



ORACLE
NetSuite

Bank Statement Parser Plug-in

2025.2

September 3, 2025



Copyright © 2005, 2025, Oracle and/or its affiliates.

This software and related documentation are provided under a license agreement containing restrictions on use and disclosure and are protected by intellectual property laws. Except as expressly permitted in your license agreement or allowed by law, you may not use, copy, reproduce, translate, broadcast, modify, license, transmit, distribute, exhibit, perform, publish, or display any part, in any form, or by any means. Reverse engineering, disassembly, or decompilation of this software, unless required by law for interoperability, is prohibited.

The information contained herein is subject to change without notice and is not warranted to be error-free. If you find any errors, please report them to us in writing.

If this is software, software documentation, data (as defined in the Federal Acquisition Regulation), or related documentation that is delivered to the U.S. Government or anyone licensing it on behalf of the U.S. Government, then the following notice is applicable:

U.S. GOVERNMENT END USERS: Oracle programs (including any operating system, integrated software, any programs embedded, installed, or activated on delivered hardware, and modifications of such programs) and Oracle computer documentation or other Oracle data delivered to or accessed by U.S. Government end users are "commercial computer software," "commercial computer software documentation," or "limited rights data" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, reproduction, duplication, release, display, disclosure, modification, preparation of derivative works, and/or adaptation of i) Oracle programs (including any operating system, integrated software, any programs embedded, installed, or activated on delivered hardware, and modifications of such programs), ii) Oracle computer documentation and/or iii) other Oracle data, is subject to the rights and limitations specified in the license contained in the applicable contract. The terms governing the U.S. Government's use of Oracle cloud services are defined by the applicable contract for such services. No other rights are granted to the U.S. Government.

This software or hardware is developed for general use in a variety of information management applications. It is not developed or intended for use in any inherently dangerous applications, including applications that may create a risk of personal injury. If you use this software or hardware in dangerous applications, then you shall be responsible to take all appropriate fail-safe, backup, redundancy, and other measures to ensure its safe use. Oracle Corporation and its affiliates disclaim any liability for any damages caused by use of this software or hardware in dangerous applications.

Oracle®, Java, and MySQL are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

Intel and Intel Inside are trademarks or registered trademarks of Intel Corporation. All SPARC trademarks are used under license and are trademarks or registered trademarks of SPARC International, Inc. AMD, Epyc, and the AMD logo are trademarks or registered trademarks of Advanced Micro Devices. UNIX is a registered trademark of The Open Group.

This software or hardware and documentation may provide access to or information about content, products, and services from third parties. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to third-party content, products, and services unless otherwise set forth in an applicable agreement between you and Oracle. Oracle Corporation and its affiliates will not be responsible for any loss, costs, or damages incurred due to your access to or use of third-party content, products, or services, except as set forth in an applicable agreement between you and Oracle.

If this document is in public or private pre-General Availability status:

This documentation is in pre-General Availability status and is intended for demonstration and preliminary use only. It may not be specific to the hardware on which you are using the software. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to this documentation and will not be responsible for any loss, costs, or damages incurred due to the use of this documentation.

If this document is in private pre-General Availability status:

The information contained in this document is for informational sharing purposes only and should be considered in your capacity as a customer advisory board member or pursuant to your pre-General Availability trial agreement only. It is not a commitment to deliver any material, code, or functionality, and should not be relied upon in making purchasing decisions. The development, release, timing, and pricing of any features or functionality described in this document may change and remains at the sole discretion of Oracle.

This document in any form, software or printed matter, contains proprietary information that is the exclusive property of Oracle. Your access to and use of this confidential material is subject to the terms and conditions of your Oracle Master Agreement, Oracle License and Services Agreement, Oracle PartnerNetwork Agreement, Oracle distribution agreement, or other license agreement which has been executed by you and Oracle and with which you agree to comply. This document and information contained herein may not be disclosed, copied, reproduced, or distributed to anyone outside Oracle without prior written consent of Oracle. This document is not part of your license agreement nor can it be incorporated into any contractual agreement with Oracle or its subsidiaries or affiliates.

Documentation Accessibility

For information about Oracle's commitment to accessibility, visit the Oracle Accessibility Program website at <https://www.oracle.com/corporate/accessibility>.

Access to Oracle Support

Oracle customers that have purchased support have access to electronic support through My Oracle Support. For information, visit <http://www.oracle.com/pls/topic/lookup?ctx=acc&id=info> or visit <http://www.oracle.com/pls/topic/lookup?ctx=acc&id=trs> if you are hearing impaired.

Sample Code

Oracle may provide sample code in SuiteAnswers, the Help Center, User Guides, or elsewhere through help links. All such sample code is provided "as is" and "as available", for use only with an authorized NetSuite Service account, and is made available as a SuiteCloud Technology subject to the SuiteCloud Terms of Service at www.netsuite.com/tos.

Oracle may modify or remove sample code at any time without notice.

No Excessive Use of the Service

As the Service is a multi-tenant service offering on shared databases, Customer may not use the Service in excess of limits or thresholds that Oracle considers commercially reasonable for the Service. If Oracle reasonably concludes that a Customer's use is excessive and/or will cause immediate or ongoing performance issues for one or more of Oracle's other customers, Oracle may slow down or throttle Customer's excess use until such time that Customer's use stays within reasonable limits. If Customer's particular usage pattern requires a higher limit or threshold, then the Customer should procure a subscription to the Service that accommodates a higher limit and/or threshold that more effectively aligns with the Customer's actual usage pattern.

Beta Features

This software and related documentation are provided under a license agreement containing restrictions on use and disclosure and are protected by intellectual property laws. Except as expressly permitted in your license agreement or allowed by law, you may not use, copy, reproduce, translate, broadcast, modify, license, transmit, distribute, exhibit, perform, publish, or display any part, in any form, or by any means. Reverse engineering, disassembly, or decompilation of this software, unless required by law for interoperability, is prohibited.

The information contained herein is subject to change without notice and is not warranted to be error-free. If you find any errors, please report them to us in writing.

If this is software or related documentation that is delivered to the U.S. Government or anyone licensing it on behalf of the U.S. Government, then the following notice is applicable:

U.S. GOVERNMENT END USERS: Oracle programs (including any operating system, integrated software, any programs embedded, installed or activated on delivered hardware, and modifications of such programs) and Oracle computer documentation or other Oracle data delivered to or accessed by U.S. Government end users are "commercial computer software" or "commercial computer software documentation" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, reproduction, duplication, release, display, disclosure, modification, preparation of derivative works, and/or adaptation of i) Oracle programs (including any operating system, integrated software, any programs embedded, installed or activated on delivered hardware, and modifications of such programs), ii) Oracle computer documentation and/or iii) other Oracle data, is subject to the rights and limitations specified in the license contained in the applicable contract. The terms governing the U.S. Government's use of Oracle cloud services are defined by the applicable contract for such services. No other rights are granted to the U.S. Government.

This software or hardware is developed for general use in a variety of information management applications. It is not developed or intended for use in any inherently dangerous applications, including applications that may create a risk of personal injury. If you use this software or hardware in dangerous applications, then you shall be responsible to take all appropriate fail-safe, backup, redundancy, and other measures to ensure its safe use. Oracle Corporation and its affiliates disclaim any liability for any damages caused by use of this software or hardware in dangerous applications.

Oracle and Java are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

Intel and Intel Inside are trademarks or registered trademarks of Intel Corporation. All SPARC trademarks are used under license and are trademarks or registered trademarks of SPARC International, Inc. AMD, Epyc, and the AMD logo are trademarks or registered trademarks of Advanced Micro Devices. UNIX is a registered trademark of The Open Group.

This software or hardware and documentation may provide access to or information about content, products, and services from third parties. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to third-party content, products, and services unless otherwise set forth in an applicable agreement between you and Oracle. Oracle Corporation and its affiliates will not be responsible for any loss, costs, or damages incurred due to your access to or use of third-party content, products, or services, except as set forth in an applicable agreement between you and Oracle.

This documentation is in pre-General Availability status and is intended for demonstration and preliminary use only. It may not be specific to the hardware on which you are using the software. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to this documentation and will not be responsible for any loss, costs, or damages incurred due to the use of this documentation.

The information contained in this document is for informational sharing purposes only and should be considered in your capacity as a customer advisory board member or pursuant to your pre-General Availability trial agreement only. It is not a commitment to deliver any material, code, or functionality, and should not be relied upon in making purchasing decisions. The development, release, and timing of any features or functionality described in this document remains at the sole discretion of Oracle.

This document in any form, software or printed matter, contains proprietary information that is the exclusive property of Oracle. Your access to and use of this confidential material is subject to the terms and conditions of your Oracle Master Agreement, Oracle License and Services Agreement, Oracle PartnerNetwork Agreement, Oracle distribution agreement, or other license agreement which has been executed by you and Oracle and with which you agree to comply. This document and information contained herein may not be disclosed, copied, reproduced, or distributed to anyone outside Oracle without prior written consent of Oracle. This document is not part of your license agreement nor can it be incorporated into any contractual agreement with Oracle or its subsidiaries or affiliates.

Table of Contents

Bank Statement Parser Plug-in Interface Overview	1
Bank Statement Parser Process Flow	2
Developing a Bank Statement Parser Plug-in	2
Enabling Features for a Bank Statement Parser Plug-in	2
Creating a Bank Statement Parser Plug-in Script File	3
Testing the Bank Statement Parser Plug-in	5
Administering a Bank Statement Parser Plug-in	5
Enabling Features for a Bank Statement Parser Plug-in	5
Adding the Plug-in Implementation	5
Activating a Bank Statement Parser Plug-in	6
Bank Statement Parser Plug-in Interface Definition	8
parseBankStatement	9
BankStatementParserInput	10
BankStatementParserOutput	12
getStandardTransactionCodes	25
StandardTransactionCodeOutput	25

Bank Statement Parser Plug-in Interface Overview

 **Warning:** The Bank Statement Parser Plug-in interface is deprecated as of NetSuite 2020.2. This means that the plug-in interface is still functional, but is no longer supported. To parse and retrieve additional data types, including corporate card expense data for expense reporting, use the Financial Institution Parser Plug-in interface. For details, see the help topic [Financial Institution Parser Plug-in Interface Overview](#).

Use a Bank Statement Parser Plug-in to create and upload your own parsers for different bank and credit card statement formats and store them in NetSuite. Bank Statement Parser Plug-ins translate the information from file formats that are not supported by NetSuite's default parsers. The Bank Statement Parser Plug-ins then turn the data into account statements or transactions that NetSuite can understand and use. This enables you to import other file formats and then use downstream matching and reconciliation features.

To create a Bank Statement Parser Plug-in, you need to create the plug-in implementation script file, and then upload the script file and other utility files as required. For details, see [Creating a Bank Statement Parser Plug-in Script File](#).

You can define alternative implementations that suit your needs by customizing the plug-in's logic. To accommodate this process, NetSuite provides an interface definition that describes the name, parameters, and return type of each function defined in the plug-in's interface. For details, see [Bank Statement Parser Plug-in Interface Definition](#).

A Bank Statement Parser Plug-in can be used for both manual import and automatic import of bank data:

For manual import:

- NetSuite administrators associate a Bank Statement Parser Plug-in with a format profile for the appropriate financial institution record. For details, see the help topic [Format Profile Creation](#)
- NetSuite users import a bank statement from the Upload File page (previously named Statement File Import), using a Bank Statement Parser Plug-in and configuration settings from an existing format profile. They can manually upload the file, or they can forgo a file upload by using the Bank Connectivity Plug-in. For details, see the help topic [Manual Bank Data Import](#).

For automatic import, administrators must configure a format profile that uses a Bank Statement Parser Plug-in with a Financial Institution Connectivity Plug-in. These two components enable them to directly connect to a financial institution and automate the retrieval of bank data. For details, see the help topics [Automatic Bank Data Import](#) and [Format Profile Creation](#). For information on the Financial Institution Connectivity Plug-in, see the help topic [Financial Institution Connectivity Plug-in Interface Overview](#).

A Bank Statement Parser Plug-in is supported by the SuiteCloud Development Framework (SDF). SDF enables you to customize your NetSuite account from an integrated development environment (IDE) on your local computer. For details, see .

You can customize a Bank Statement Parser Plug-in in SDF by defining bankstatementparserplugin objects. For details, see the help topic [Bank Statement Parser Plug-ins as XML Definitions](#).

NetSuite Role	Topic Title
All roles	Bank Statement Parser Process Flow
Developer	Developing a Bank Statement Parser Plug-in

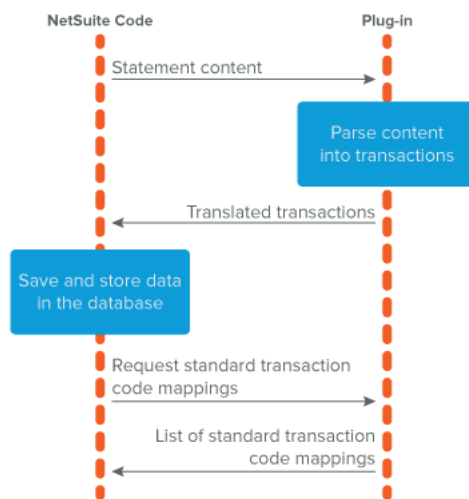
NetSuite Role	Topic Title
Administrator	Administering a Bank Statement Parser Plug-in



Important: The Bank Statement Parser Plug-in uses SuiteScript 2.0. Your entry point scripts and supporting library scripts must also use SuiteScript 2.0. For information, see the help topics [SuiteScript 2.x Script Creation Process](#) and [SuiteScript 2.x](#).

Bank Statement Parser Process Flow

The following diagram shows the workflow for a Bank Statement Parser Plug-in:



Developing a Bank Statement Parser Plug-in

To develop a Bank Statement Parser Plug-in, complete the following steps:

- [Enabling Features for a Bank Statement Parser Plug-in](#)
- [Creating a Bank Statement Parser Plug-in Script File](#)
- [Testing the Bank Statement Parser Plug-in](#)

Enabling Features for a Bank Statement Parser Plug-in

To begin development or administration for a Bank Statement Parser Plug-in, make sure that the Server SuiteScript feature is enabled on the development account.

To enable features for the plug-in:

1. Choose Setup > Company > Enable Features.
2. On the **SuiteCloud** subtab, make sure **Server SuiteScript** is checked. If necessary, check the box and agree to the Terms of Service.

3. Click **Save**.

Creating a Bank Statement Parser Plug-in Script File

You must implement each Bank Statement Parser Plug-in interface function in a Javascript file (with a .js extension). You can use the SuiteCloud IDE, another Javascript IDE, or a text editor to create the plug-in implementation script file.

This plug-in uses SuiteScript 2.0. Begin your implementation script file with this comment:

The following is a parser sample:

```

1  /**
2   * @ApiVersion 2.0
3   * @NScriptType bankStatementParserPlugin
4   */
5
6  define(['N/file', 'N/log'],
7    function(file, log) {
8      return {
9        parseBankStatement: function (context) {
10
11          var accountStatement = context.output.createNewAccountStatement();
12          accountStatement.accountMappingKey = defaultAccountMappingKey;
13          log.debug({
14            title: 'Adding a new account statement',
15            details: accountStatement
16          });
17
18          var accountStatementId = context.output.addAccountStatement({"parsedAccountStatement":accountStatement});
19          log.debug({
20            title: 'New account statement ID',
21            details: accountStatementId
22          });
23
24          var statementFile = context.input.file;
25
26          var statementLineIterator = statementFile.lines.iterator();
27          statementLineIterator.next();
28          statementLineIterator.each(function (line) {
29            log.debug({
30              title: 'Read a line from the statement',
31              details: line.value
32            });
33
34            var formatProfileId = context.input.formatProfileId;
35            log.debug({
36              title: 'Format Profile Id',
37              details: formatProfileId
38            });
39
40            var partsOfCSVLine = line.value.split(',');
41            var transaction = context.output.createNewTransaction();
42            transaction.accountStatementId = accountStatementId;
43            var rawDate = partsOfCSVLine[0];
44            transaction.date = rawDate.substring(6, 10) + '-' + rawDate.substring(0, 2) + '-' + rawDate.substring(3, 5);
45            transaction.amount = partsOfCSVLine[4];
46            transaction.transactionMappingKey = partsOfCSVLine[3];
47            transaction.transactionNumber = partsOfCSVLine[2];
48            transaction.payee = partsOfCSVLine[1];
49            transaction.currency = "USD";
50            transaction.memo = partsOfCSVLine[5];
51            transaction.customerRawId = partsOfCSVLine[6];

```



```

51         transaction.customerName = partsOfCSVLine[7];
52         transaction.invoices = partsOfCSVLine[8].split(',');
53
54         log.debug({
55             title: 'Adding a new transaction',
56             details: transaction
57         });
58
59         context.output.addTransaction({"parsedTransaction":transaction});
60         return true;
61     });
62
63 },
64 getStandardTransactionCodes: function (context) {
65     var tranTypes = ["ACH", "CHECK", "CREDIT", "DEBIT", "DEPOSIT", "FEE", "INTEREST", "PAYMENT", "TRANSFER", "OTHER"];
66     for (var i = 0; i < tranTypes.length; ++i)
67     {
68         var standardTransactionCode = context.output.createNewStandardTransactionCode();
69         standardTransactionCode.tranCode = tranTypes[i];
70         standardTransactionCode.tranType = tranTypes[i];
71         context.output.addStandardTransactionCode({"standardTransactionCode":standardTransactionCode});
72
73         log.debug({
74             title: 'Adding a new standard transaction code',
75             details: standardTransactionCode
76         });
77     }
78 }
79 }
80 }
81 );

```

To surface user-facing error messages that are readable, you must raise error messages as simple strings in the plug-in script. Currently, throwing error objects generates errors as JSON strings to users, which are not easily readable. For examples of raising an error message as a simple string, see the sample strings below:

```
1 | throw('The file could not be imported because the currency code is not valid.');
```

```
1 | var translateableErrorMessage = translation.get({collection: 'my_errors', key: 'CURRENCY_ERROR'}()); throw(translateableErrorMessage);
```

In the second example, **translation.get** is a SuiteScript API that can be implemented by plug-in authors if they wish to translate strings. For more information about translation, see the help topic [N/translation Module](#).

Your plug-in script should contain all logic required for your implementation. The following table describes the functions that you **must** implement in the plug-in script file. For details about the required functions and their objects, see [Bank Statement Parser Plug-in Interface Definition](#).

The following table describes the interface functions:

Function	Description
parseBankStatement (BankStatementParserContext)	Parse a streamed bank statement. This function can stream bank statement file content and obtain default settings for the file. This function adds the parsed transactions and account statements to NetSuite. BankStatementParserContext is the context for this function.
getStandardTransactionCodes (StandardTransactionCodeContext)	Store the standard transaction code mapping in NetSuite.

Function	Description
	This function contains information that maps the transaction code from the parsed transactions to the corresponding NetSuite transaction types. StandardTransactionCodeContext is the context for this function.

Rules and Guidelines for Creating a Plug-in Implementation Script File

Use the following rules and guidelines when creating the plug-in implementation script file:

- The plug-in script file can have any name, as long as it has a .js extension and contains an implementation of each of the interface functions.
- If you want to create utility files with helper functions to use with the plug-in implementation script file, you can include those files when you create the plug-in implementation in NetSuite.
- Due to limitations with how strings are handled in NetSuite, you cannot use the JavaScript case statement in the plug-in implementation script file.
- The plug-in allows up to 1000 usage units.
- There are special character restrictions set out by the import file specifications. For example, according to the OFX specification, the characters "<", ">", and "&" must be escaped using the entities "<", ">", and "&", respectively. Otherwise, the parser will fail during the import process. If an uploaded file violates the file format specification for its file type, the parser will throw an error.
- If the character limit is exceeded in a field, the value will be truncated.

Testing the Bank Statement Parser Plug-in

To enable the plug-in for testing, see [Adding the Plug-in Implementation](#).

Administering a Bank Statement Parser Plug-in

To install and set up a Bank Statement Parser Plug-in, complete the following steps:

- [Enabling Features for a Bank Statement Parser Plug-in](#)
- [Adding the Plug-in Implementation](#)
- [Activating a Bank Statement Parser Plug-in](#)

Enabling Features for a Bank Statement Parser Plug-in

See [Enabling Features for a Bank Statement Parser Plug-in](#).

Adding the Plug-in Implementation

After creating the plug-in script file, upload and implement the file in NetSuite.

To add the plug-in implementation:

1. In NetSuite, go to Customization > Plug-ins > Plug-in Implementations > New.
NetSuite displays the Upload Plug-in Implementation page.
2. For the **Script File**, click the plus icon.
3. Click **Choose File**, then select the JavaScript file.
4. Click **Save**.
5. Click **Create Plug-in Implementation**.
6. On the **Plug-in Implementation** page, enter the following information:

Option	Description
Name	User-friendly name for the implementation. This name appears in the plug-in implementation list.
ID	Internal ID for the implementation for use in scripting. If you do not provide an ID, NetSuite will provide one for you after you click Save . As a developer, when you write code that uses this implementation, you will reference the implementation using this ID.
Status	Current status for the implementation. Choose Testing to have the implementation accessible to the owner of the implementation. Choose Released to have the implementation accessible in a production environment
Log Level	Logging level you want for the script. Select Debug, Audit, Error, or Emergency. The log information appears on the Execution Log subtab for the implementation after you create it. Go to Customization > Plug-ins > Plug-in Implementations, select your implementation, and then click the Execution Log subtab.
Execute As Role	Role that the script runs as. The Execute As Role field provides role-based granularity in terms of the permissions and restrictions of the executing script. The Current Role value indicates that the script executes with the permissions of the currently logged-in NetSuite user. If you created a custom role, be sure to test using that role. You can then bundle the custom role to distribute it with the plug-in implementation.
Description	Optional description of the implementation.
Owner	User account that owns the implementation. Default is the name of the current user
Inactive	Indicates that the plug-in implementation does not run in the account. Inactivate a plug-in implementation, for example, to temporarily disable it for testing purposes.

7. On the **Unhandled Errors** subtab, select which individual(s) will be notified if script errors occur. By default the **Notify Script Owner** box is checked.

To enter multiple email addresses in the **Notify Emails** box, separate email addresses with a semi-colon.
8. Click **Save**.

Activating a Bank Statement Parser Plug-in

After you install the plug-in bundle, you can activate the plug-in. The developer must give you the name of the plug-in implementation if it is different from the name of the SuiteApp.

To activate the plug-in:

1. To activate the plug-in implementation, go to Customization > Plug-ins > Manage Plug-ins.
2. Under **Bank Statement Parser**, check the box next to the name of your plug-in implementation.

3. Click **Save**.

You can access the list of implementations by going to Customization > Plug-ins > Plug-in Implementations.

Bank Statement Parser Plug-in Interface Definition

Warning: The Bank Statement Parser Plug-in interface is deprecated as of NetSuite 2020.2. This means that the plug-in interface is still functional, but is no longer supported. To parse and retrieve additional data types, including corporate card expense data for expense reporting, use the Financial Institution Parser Plug-in interface. See the help topics [Financial Institution Parser Plug-in Interface Overview](#) and [Financial Institution Parser Plug-in Interface Definition](#).

This chapter describes the functions, objects, and methods for the Bank Statement Parser Plug-in.

Plug-in Interface Functions

The plug-in interface includes the following functions.

Function	Description
parseBankStatement (BankStatementParserContext)	Parse a streamed bank statement. This function can stream bank statement file content and obtain default settings for the file. This function adds the parsed transactions and account statements to NetSuite. When NetSuite requests the bank statement content, the plug-in implements this function, which parses and sends the translated transaction data to NetSuite. BankStatementParserContext is the context for this function.
getStandardTransactionCodes (StandardTransactionCodeContext)	Store the standard transaction code mapping to NetSuite. This function contains information that maps the transaction code from the parsed transactions to the corresponding NetSuite transaction types. When NetSuite requests the standard transaction code mapping, the plug-in implements this function, which sends the list back to NetSuite. StandardTransactionCodeContext is the context for this function.

Context Objects

The Bank Statement Parser Plug-in interface functions use the following context objects.

Object	Description
BankStatementParserContext	Input – BankStatementParserInput Output – BankStatementParserOutput parseBankStatement uses this context object.
StandardTransactionCodeContext	Output – StandardTransactionCodeOutput getStandardTransactionCodes uses this context object.

parseBankStatement

Function Declaration void parseBankStatement(BankStatementParserContext bankStatementParserContext)

Type Interface function

Description Parse a streamed bank statement.

This function can stream bank statement file content and obtain default settings for the file. This function adds the parsed transactions and account statements to NetSuite.

When NetSuite requests the bank statement content, the plug-in implements this function, which parses and sends the translated transaction data to NetSuite.

Returns void

Input Parameters bankStatementParserContext

Example

```

1      parseBankStatement: function (context) {
2
3          var accountStatement = context.output.createNewAccountStatement();
4          accountStatement.accountMappingKey = defaultAccountMappingKey;
5          log.debug({
6              title: 'Adding a new account statement',
7              details: accountStatement
8          });
9
10         var accountStatementId = context.output.addAccountStatement({"parsedAccountStatement":accountStatement});
11         log.debug({
12             title: 'New account statement ID',
13             details: accountStatementId
14         });
15
16         var statementFile = context.input.file;
17
18         var statementLineIterator = statementFile.lines.iterator();
19         statementLineIterator.next();
20         statementLineIterator.each(function (line) {
21             log.debug({
22                 title: 'Read a line from the statement',
23                 details: line.value
24             });
25
26             var partsOfCSVLine = line.value.split(',');
27             var transaction = context.output.createNewTransaction();
28             transaction.accountStatementId = accountStatementId;
29             var rawDate = partsOfCSVLine[0];
30             transaction.date = rawDate.substring(6, 10) + '-' + rawDate.substring(0, 2) + '-' + rawDate.substring(3, 5);
31             transaction.amount = partsOfCSVLine[4];
32             transaction.transactionMappingKey = partsOfCSVLine[3];
33             transaction.transactionNumber = partsOfCSVLine[2];
34             transaction.payee = partsOfCSVLine[1];
35             transaction.currency = "USD";
36             transaction.memo = partsOfCSVLine[5];
37             transaction.customerRawId = partsOfCSVLine[6];
38             transaction.customerName = partsOfCSVLine[7];
39             transaction.invoices = partsOfCSVLine[8].split(',');
40
41             log.debug({
42                 title: 'Adding a new transaction',
43                 details: transaction
44             });
45
46             context.output.addTransaction({"parsedTransaction":transaction});
47             return true;

```

```

48 |         });
49 |
50 |     },

```

BankStatementParserInput

Type	Interface input object
Description	Input object for parseBankStatement . When NetSuite requests the bank statement content, this object obtains the bank statement file content and default settings from the file.
Methods	■ getFile ■ getFormatProfileId
Parent Object(s)	N/A
Child Object(s)	N/A

getFile

Function Declaration	nlobjFileInterface getFile()
Type	Object method
Description	This method gets the imported file content to the parser plug-in.
Returns	nlobjFileInterface – A file object containing the bank statement content. You can also use a file API to stream large files. For more information on streaming large files, see the help topic N/file Module .
Input Parameters	None
Parent object	BankStatementParserInput

Example

```

1 |     var statementFile = context.input.file;
2 |
3 |     var statementLineIterator = statementFile.lines.iterator();
4 |     statementLineIterator.next();
5 |     statementLineIterator.each(function (line) {
6 |         log.debug({
7 |             title: 'Read a line from the statement',
8 |             details: line.value
9 |         });

```

getFormatProfileId

Function Declaration	int getFormatProfileId()
Type	Object method

Description	This method passes the ID of the format profile, which the user selected at the time of import. To get related configurations to parse the bank or credit card statement, the script can use this ID combined with SuiteScript. These configurations could include date format, currency precision, and other user-defined configurations.
Returns	int – the ID of the format profile, which the user selected at the time of import.
Input Parameters	None
Parent object	BankStatementParserInput

Example

The following example is a generic implementation of the getFormatProfileId method:

```

1      var formatProfileId = context.input.formatProfileId;
2      log.debug({
3          title: 'Format Profile Id',
4          details: formatProfileId
5      });

```

The following example uses the getFormatProfileId method to get parser configuration values from a format profile record:

```

1  function getParserConfigs(context) {
2      var ftid = context.input.formatProfileId;
3      var ppid = 'customscript_bsp_parser_bai2';
4      var configs = {};
5
6      search.create({
7          type : 'customrecord_bsp_configuration_value',
8          filters : [
9              ['custrecord_bsp_config_ftid', search.Operator.IS, ftid],
10             'and',
11             ['custrecord_bsp_config_ppid', search.Operator.IS, ppid]
12         ],
13         columns : [
14             'name',
15             'custrecord_bsp_config_value'
16         ]
17     }).run().each(function (result) {
18         var name = result.getValue('name');
19         var value = result.getValue('custrecord_bsp_config_value');
20
21         configs[name] = {
22             id : result.id,
23             value : value
24         };
25
26         return true;
27     });
28
29     return configs;
30 };

```

In the preceding example, the custom record customrecord_bsp_configuration_value has a foreign key called custrecord_bsp_config_ftid that connects it to the format profile. A User Event script is deployed to display these records in the Configuration subtab on the Format Profile page, where users set their values.

The function getParserConfigs is triggered when the import process starts. When triggered, the function locates the configurations necessary for parsing the imported statement, according to the specified file format. In the preceding example, customscript_bsp_parser_bai2 stands for BAI2.

BankStatementParserOutput

Type	Interface output object
Description	Output object for parseBankStatement . When NetSuite requests the bank statement content, this object parses and sends the translated transaction data to NetSuite.
Methods	■ createNewAccountStatement ■ addAccountStatement ■ createNewTransaction ■ addTransaction
Parent Object(s)	N/A
Child Object(s)	■ ParsedAccountStatement ■ ParsedTransaction

createNewAccountStatement

Function Declaration	<code>ParsedAccountStatement createNewAccountStatement()</code>
Type	Object method
Description	Create a new ParsedAccountStatement object to enable the Bank Statement Parser Plug-in to set property values. This method does not persist a new account statement in NetSuite.
Returns	ParsedAccountStatement
Input Parameters	None
Parent object	BankStatementParserOutput

Example

```

1 | var accountStatement = context.output.createNewAccountStatement();
2 | accountStatement.accountMappingKey = defaultAccountMappingKey;
3 | log.debug({
4 |     title: 'Adding a new account statement',
5 |     details: accountStatement
6 | });

```

addAccountStatement

Function Declaration	<code>int addAccountStatement(ParsedAccountStatement parsedAccountStatement)</code>
Type	Object method
Description	The plug-in calls this method to persist an account statement record in NetSuite, which enables the plug-in to later associate transactions to the account.
Returns	int – the record ID of the account statement.

Input Parameters [ParsedAccountStatement](#) – the object containing information to create the account statement record.

Parent object [BankStatementParserOutput](#)

Example

```

1      var accountStatementId = context.output.addAccountStatement({"parsedAccountStatement":accountStatement});
2      log.debug({
3          title: 'New account statement ID',
4          details: accountStatementId
5      });

```

createNewTransaction

Function Declaration `ParsedTransaction createNewTransaction()`

Type Object method

Description Create a new object handler to enable the Bank Statement Parser Plug-in to set property values.

This method does not persist a new transaction in NetSuite.

Returns [ParsedTransaction](#)

Input Parameters None

Parent object [BankStatementParserOutput](#)

Example

```

1      var partsOfCSVLine = line.value.split(',');
2      var transaction = context.output.createNewTransaction();
3      transaction.accountStatementId = accountStatementId;
4      var rawDate = partsOfCSVLine[0];
5      transaction.date = rawDate.substring(6, 10) + '-' + rawDate.substring(0, 2) + '-' + rawDate.substring(3, 5);
6      transaction.amount = partsOfCSVLine[4];
7      transaction.transactionMappingKey = partsOfCSVLine[3];
8      transaction.transactionNumber = partsOfCSVLine[2];
9      transaction.payee = partsOfCSVLine[1];
10     transaction.currency = "USD";
11     transaction.memo = partsOfCSVLine[5];
12     transaction.customerRawId = partsOfCSVLine[6];
13     transaction.customerName = partsOfCSVLine[7];
14     transaction.invoices = partsOfCSVLine[8].split(',');

```

addTransaction

Function Declaration `void addTransaction(ParsedTransaction parsedTransaction)`

Type Object method

Description The plug-in calls this method to persist the new transaction in NetSuite.

Returns void

Input Parameters [ParsedTransaction](#) – the new transaction to be persisted to NetSuite.

Parent object [BankStatementParserOutput](#)

Example

```
1 | context.output.addTransaction({"parsedTransaction":transaction});
2 |   return true;
3 |   };
```

ParsedAccountStatement

Type	Child Object
Description	Definition of a parsed account statement. createNewAccountStatement uses this object.
Methods	■ getAccountMappingKey ■ setAccountMappingKey ■ getStatementDate ■ setStatementDate ■ getOpeningBalance ■ setOpeningBalance ■ getClosingBalance ■ setClosingBalance
Parent Object(s)	BankStatementParserOutput
Child Object(s)	N/A

getAccountMappingKey

Function Declaration	string getAccountMappingKey()
Type	Object method
Description	This method gets the account mapping key, to associate the parsed account statements with a selected NetSuite account.
Returns	string
Input Parameters	None
Parent object	ParsedAccountStatement

setAccountMappingKey

Function Declaration	void setAccountMappingKey(string accountMappingKey)
Type	Object method
Description	This method sets the account mapping key. The account mapping key must map to a valid NetSuite account ID. The mapping can be set up in the format profile or in the Online Banking Data Upload page. This is a required field.
Returns	void

Input Parameters	accountMappingKey – The value is used to map the account statements to a NetSuite account.
Parent object	ParsedAccountStatement

getStatementDate

Function Declaration	string getStatementDate()
Type	Object method
Description	This method gets the statement date.
Returns	string
Input Parameters	None
Parent object	ParsedAccountStatement

setStatementDate

Function Declaration	void setStatementDate(string statementDate)
Type	Object method
Description	This method sets the account statement date. When set, the following fields are populated from the most recent imported account statement containing a statement date: ■ Bank Balance and Balance As Of on the Match Bank Data page ■ Statement End Date and Ending Statement Balance on the Reconcile Account Statement page If no account statements contain a statement date, the following takes place: ■ The Bank Balance and Balance As Of fields are not populated. ■ The Statement End Date and Ending Statement Balance fields are populated with the current time, and the closing balance from the last reconciliation statement, respectively.  Note: If you use a default parser to upload a file without an account statement date, NetSuite uses the most recent imported transaction date to set the statement date. This date may or may not correspond to the real account statement date. The date must be in the format YYYY-MM-DD. For example, "2018-05-28".
Returns	void
Input Parameters	statementDate – the statement date
Parent object	ParsedAccountStatement

getOpeningBalance

Function Declaration	string getOpeningBalance()
Type	Object method
Description	This method gets the opening balance for the statement.

Returns	string
Input Parameters	None
Parent object	ParsedAccountStatement

setOpeningBalance

Function Declaration	void setOpeningBalance(string openingBalance)
Type	Object method
Description	This method sets the opening balance for the statement.
Returns	void
Input Parameters	openingBalance
Parent object	ParsedAccountStatement

getClosingBalance

Function Declaration	string getClosingBalance()
Type	Object method
Description	This method gets the closing balance for the statement.
Returns	string
Input Parameters	None
Parent object	ParsedAccountStatement

setClosingBalance

Function Declaration	void setClosingBalance(string closingBalance)
Type	Object method
Description	This method sets the closing balance for the statement.
Returns	void
Input Parameters	closingBalance
Parent object	ParsedAccountStatement

ParsedTransaction

Type	Child Object
Description	Definition of a parsed transaction. createNewTransaction uses this object.
Methods	■ getAccountStatementId

- [setAccountStatementId](#)
- [getDate](#)
- [setDate](#)
- [getAmount](#)
- [setAmount](#)
- [getTransactionMappingKey](#)
- [setTransactionMappingKey](#)
- [getTransactionNumber](#)
- [setTransactionNumber](#)
- [getPayee](#)
- [setPayee](#)
- [getCurrency](#)
- [setCurrency](#)
- [getMemo](#)
- [setMemo](#)
- [getUniqueId](#)
- [setUniqueId](#)
- [getSourceId](#)
- [setSourceId](#)
- [getTransactionStatus](#)
- [setTransactionStatus](#)
- [getCustomerRawId](#)
- [setCustomerRawId](#)
- [getCustomerName](#)
- [setCustomerName](#)
- [getInvoices](#)
- [setInvoices](#)

Parent Object(s) [BankStatementParserOutput](#)

Child Object(s) N/A

getAccountStatementId

Function Declaration	<code>int getAccountStatementId()</code>
Type	Object method
Description	This method gets the account statement ID.
Returns	int
Input Parameters	None
Parent object	ParsedTransaction

setAccountStatementId

Function Declaration `void setAccountStatementId(integer accountStatementId)`

Type	Object method
Description	This method sets the account statement ID, to which the transaction is associated. This record ID must belong to a valid account statement record, which NetSuite creates during the bank import process. This is a required field.
Returns	void
Input Parameters	accountStatementId – the record ID of the account statement.
Parent object	ParsedTransaction

getDate

Function Declaration	string getDate()
Type	Object method
Description	This method gets the date for the transaction.
Returns	string
Input Parameters	None
Parent object	ParsedTransaction

setDate

Function Declaration	void setDate(string date)
Type	Object method
Description	This method sets the date for the transaction. The date must be in the format YYYY-MM-DD. For example, "2018-05-28". This is a required field.
Returns	void
Input Parameters	date – the date for the transaction.
Parent object	ParsedTransaction

getAmount

Function Declaration	string getAmount()
Type	Object method
Description	This method gets the amount for the transaction. In bank and credit card statements, amounts must be provided in decimal formats only, with no commas. For example, "1 000 000.00". Negative numbers must be represented with a minus (-) sign. For example, "-100".
Returns	string

Input Parameters	None
Parent object	ParsedTransaction

setAmount

Function Declaration	void setAmount(string amount)
Type	Object method
Description	This method sets the amount for the transaction. This is a required field.
Returns	void
Input Parameters	amount – the amount for the transaction.
Parent object	ParsedTransaction

getTransactionMappingKey

Function Declaration	string getTransactionMappingKey()
Type	Object method
Description	This method gets the transaction mapping key.
Returns	string
Input Parameters	None
Parent object	ParsedTransaction

setTransactionMappingKey

Function Declaration	void setTransactionMappingKey(string transactionMappingKey)
Type	Object method
Description	This method sets the transaction mapping key. This transaction mapping key must map to a valid NetSuite transaction type. The mapping can be set up in the format profile or in getStandardTransactionCodes . This is a required field.
Returns	void
Input Parameters	transactionMappingKey – the mapping key.
Parent object	ParsedTransaction

getTransactionNumber

Function Declaration	string getTransactionNumber()
Type	Object method

Description	This method gets the transaction number.
Returns	string
Input Parameters	None
Parent object	ParsedTransaction

setTransactionNumber

Function Declaration	<code>void setTransactionNumber(string transactionNumber)</code>
Type	Object method
Description	This method sets the transaction number.
Returns	void
Input Parameters	transactionNumber
Parent object	ParsedTransaction

getPayee

Function Declaration	<code>string getPayee()</code>
Type	Object method
Description	This method gets the payee for the transaction.
Returns	string
Input Parameters	None
Parent object	ParsedTransaction

setPayee

Function Declaration	<code>void setPayee(string payee)</code>
Type	Object method
Description	This method sets the payee for the transaction.
Returns	void
Input Parameters	payee
Parent object	ParsedTransaction

getCurrency

Function Declaration	<code>string getCurrency()</code>
Type	Object method
Description	This method gets the currency code for the transaction.

Returns	string
Input Parameters	None
Parent object	ParsedTransaction

setCurrency

Function Declaration	void setCurrency(string currency)
Type	Object method
Description	This method sets the currency code for the transaction. The currency must be one of the standard currency codes.
Returns	void
Input Parameters	currency – the currency code.
Parent object	ParsedTransaction

getMemo

Function Declaration	string getMemo()
Type	Object method
Description	This method gets the memo for the transaction, if applicable.
Returns	string
Input Parameters	None
Parent object	ParsedTransaction

setMemo

Function Declaration	void setMemo(string memo)
Type	Object method
Description	This method sets the memo for the transaction, if applicable.
Returns	void
Input Parameters	memo
Parent object	ParsedTransaction

getUniqueId

Function Declaration	string getUniqueId()
Type	Object method
Description	This method gets the unique ID for the transaction.

Returns	string
Input Parameters	None
Parent object	ParsedTransaction

setUniqueId

Function Declaration	void setUniqueId(string uniqueId)
Type	Object method
Description	This method sets the unique ID for the transaction. The unique ID automatically prevents a transaction from getting imported into NetSuite as a duplicate.



Note: If your transactions contain a unique ID, you should always use this method to ensure the removal of duplicates. This is the case for both manual file import and financial institution connectivity.

Returns	void
Input Parameters	uniqueId
Parent object	ParsedTransaction

getSourceId

Function Declaration	string getSourceId()
Type	Object method
Description	This method gets the source ID for the transaction.
Returns	string
Input Parameters	None
Parent object	ParsedTransaction

setSourceId

Function Declaration	void set SourceId(string sourceId)
Type	Object method
Description	This method sets the source ID for the transaction.



Note: You should only use this method for OFX files.

The source ID maps to a field that is only present in OFX files called FitId. This field is used in addition to others such as amount, date, and memo during the duplicate detection process. Alone, you cannot use this field to detect duplicate transactions in OFX files.

To guarantee the automatic detection and removal of duplicates before they are imported, regardless of the file format, see [setUniqueId](#).

Returns	void
----------------	------

Input Parameters	sourceId
Parent object	ParsedTransaction

getTransactionStatus

Function Declaration	string getTransactionStatus()
Type	Object method
Description	This method gets the transaction status.
Returns	string
Input Parameters	None
Parent object	ParsedTransaction

setTransactionStatus

Function Declaration	void setTransactionStatus(string transactionStatus)
Type	Object method
Description	This method sets the transaction status.
Returns	void
Input Parameters	transactionStatus
Parent object	ParsedTransaction

getCustomerRawId

Function Declaration	string getCustomerRawId()
Type	Object method
Description	This method gets the customer ID for the transaction.
Returns	string
Input Parameters	None
Parent object	ParsedTransaction

setCustomerRawId

Function Declaration	void setCustomerRawId(string customerRawId)
Type	Object method
Description	This method sets the customer ID for the transaction.
Returns	void
Input Parameters	customerRawId

Parent object [ParsedTransaction](#)

getCustomerName

Function Declaration `string getCustomerName()`

Type Object method

Description This method gets the customer name for the transaction.

Returns string

Input Parameters None

Parent object [ParsedTransaction](#)

setCustomerName

Function Declaration `void setCustomerName(string customerName)`

Type Object method

Description This method sets the customer name for the transaction.

Returns void

Input Parameters customerName

Parent object [ParsedTransaction](#)

getInvoices

Function Declaration `string[] getInvoices()`

Type Object method

Description This method gets the invoices associated with the transaction.

Returns string

Input Parameters None

Parent object [ParsedTransaction](#)

setInvoices

Function Declaration `void setInvoices(string[] invoices)`

Type Object method

Description This method sets the invoices associated with the transaction.

Returns void

Input Parameters invoices

Parent object [ParsedTransaction](#)

getStandardTransactionCodes

Function Declaration	<code>void getStandardTransactionCodes(StandardTransactionCodeContext standardTransactionCodeContext)</code>
Type	Interface function
Description	Store the standard transaction code mapping to NetSuite. This function contains information that maps the transaction code from the parsed transactions to the corresponding NetSuite transaction types.
Returns	void
Input Parameters	StandardTransactionCodeContext

Example

```

1      getStandardTransactionCodes: function (context) {
2          var tranTypes = ["ACH", "CHECK", "CREDIT", "DEBIT", "DEPOSIT", "FEE", "INTEREST", "PAYMENT", "TRANSFER", "OTHER"];
3          for (var i = 0; i < tranTypes.length; ++i)
4              {
5                  var standardTransactionCode = context.output.createNewStandardTransactionCode();
6                  standardTransactionCode.tranCode = tranTypes[i];
7                  standardTransactionCode.tranType = tranTypes[i];
8                  context.output.addStandardTransactionCode({"standardTransactionCode": standardTransactionCode});
9              }
10         log.debug({
11             title: 'Adding a new standard transaction code',
12             details: standardTransactionCode
13         });
14     }

```

StandardTransactionCodeOutput

Type	Interface output object
Description	Output object for getStandardTransactionCodes When NetSuite requests the standard transaction codes, this object sends the list of standard transaction code mappings to NetSuite.
Methods	■ createNewStandardTransactionCode ■ addStandardTransactionCode
Parent Object(s)	N/A
Child Object(s)	StandardTransactionCode

createNewStandardTransactionCode

Function Declaration	<code>StandardTransactionCode createNewStandardTransactionCode()</code>
Type	Object method
Description	The plug-in calls this method to create a new StandardTransactionCode object to enable the Bank Statement Parser Plug-in to set property values.

This method does not persist a new account statement in NetSuite.

Returns	StandardTransactionCode
Input Parameters	None
Parent object	StandardTransactionCodeOutput

Example

```
1 | var standardTransactionCode = context.output.createNewStandardTransactionCode();
2 | standardTransactionCode.tranCode = tranTypes[i];
3 | standardTransactionCode.tranType = tranTypes[i];
```

addStandardTransactionCode

Function Declaration	void addStandardTransactionCode(StandardTransactionCode standardTransactionCode)
Type	Object method
Description	The plug-in calls this method to pass a standard transaction code mapping to NetSuite, which enables the plug-in to match the transaction code of the parsed transaction to the correct NetSuite transaction type, and correct Credit or Debit type.
Returns	void
Input Parameters	standardTransactionCode – the object containing the mapping of standard bank transaction code to NetSuite transaction type.
Parent object	StandardTransactionCodeOutput

Example

```
1 | context.output.addStandardTransactionCode({"standardTransactionCode":standardTransactionCode});
```

StandardTransactionCode

Type	Child Object
Description	Definition of a standard transaction code. createNewStandardTransactionCode uses this object.
Methods	■ getTranCode ■ setTranCode ■ getTranType ■ setTranType ■ getCreditDebitType ■ setCreditDebitType ■ getDescription ■ setDescription
Parent object	StandardTransactionCodeOutput

Child Object(s) N/A

getTranCode

Function Declaration	string getTranCode()
Type	Object method
Description	This method gets the raw transaction code from the parsed transaction.
Returns	string
Input Parameters	None
Parent object	StandardTransactionCode

setTranCode

Function Declaration	void setTranCode(string tranCode)
Type	Object method
Description	This method sets the raw transaction code for the transaction code mapping. This is a required field.
Returns	void
Input Parameters	tranCode – raw transaction code.
Parent object	StandardTransactionCode

getTranType

Function Declaration	string getTranType()
Type	Object method
Description	This method gets the transaction type from the parsed transaction.
Returns	string
Input Parameters	None
Parent object	StandardTransactionCode

setTranType

Function Declaration	void setTranType(string tranType)
Type	Object method
Description	This method sets the NetSuite transaction type, to which the transaction code maps. This is a required field.

Returns	void
Input Parameters	tranType – Transaction type.
Parent object	StandardTransactionCode

getCreditDebitType

Function Declaration	string getCreditDebitType()
Type	Object method
Description	This method gets the type of either Credit or Debit from the parsed transaction.
Returns	string
Input Parameters	None
Parent object	StandardTransactionCode

setCreditDebitType

Function Declaration	string getCreditDebitType()
Type	Object method
Description	This method sets the type to Credit or Debit. BAI2 and other similar file formats use this method to determine if the amount should be positive or negative.
Returns	string
Input Parameters	creditDebitType – the value is either Credit or Debit.
Parent object	StandardTransactionCode

getDescription

Function Declaration	string getDescription()
Type	Object method
Description	This method gets the description from the parsed transaction.
Returns	string
Input Parameters	None
Parent object	StandardTransactionCode

setDescription

Function Declaration	void setDescription(string description)
-----------------------------	---

Type	Object method
Description	This method sets the description from the parsed transaction.
Returns	void
Input Parameters	description
Parent object	StandardTransactionCode